Emily Rick

CS580 – Intro to AI

Adversarial Search Analysis Report

November 13, 2026

This programming assignment was to use an adversarial search algorithm to write an AI program to play the board game Othello against a human player. In this version of the game, the human plays as the black chips (represented by “B” on the 8x8 board), while the AI plays as the white chips (represented by “W” on the board). The game is played via the terminal/command line interface. The goal of the game is for one party (either the human or AI player) to have the majority of their respective chip color on the board by the end of the game.

Since this program uses an adversarial search algorithm, it searches whether or not there is an opponent to play against and whether or not that opponent has the ability to change the state of the game. The algorithm has the ability to only change their states (on their turn), but not the next state (the opponent’s turn). By doing this, it can calculate strategies for recommended moves for each state. This makes the game’s outcome purely based on chance or unpredictability.

For this assignment, the algorithm needed to use a heuristic function to determine the intermediate states of the game, as well as use a minimax algorithm that utilized alpha-beta pruning. The heuristic function that I made included an intuitive design for the respective chip pieces; a design for frontier disks/chips; and the overall position of each of the chips. For the intuitive design, the program kept count of how many “W” and “B” chips were on the board at all times. At the end of the game, it totaled up how many “W” and “B” chips were on the board and declared a winner based on that number (or a tie game if both chip amounts were tied). The

frontier disks/chips' design was based on the amount of chips that were adjacent to empty tiles. Since these chips are often flipped more than others, it made sense to program the heuristic in a way to minimize the number of black chips flipped, while simultaneously maximizing the number of white chips flipped. Finally, the overall position of the chips determined the design of the heuristic. Corner tiles were easier to program as they could never be flipped, while the tiles next to the corners were much harder to implement due to the opponent often being able to control them. I programmed this by creating a static evaluation weight matrix of all of the tiles, giving corners a high positive integer value, tiles near corners a negative value, and edges a smaller positive value (such as 3, 5, or 15).

Finally, I utilized a minimax algorithm and alpha-beta pruning to implement the AI program's decision-making logic. These algorithms made the game load faster and give the minimax algorithm more time to explore more of only a specific portion of the full game tree. I implemented these two algorithms in the "ai.py" file, which gave the game's AI the ability to reduce the number of nodes it had to search through in order to find a good solution. This reduction of nodes does not change the final decision that the AI makes, as it only prunes branches that are proven to be worse than a previous node option.